Circular List

We have seen some video clippings restart from the beginning when the end is reached. There are music players which repeat the songs from the playlist. These are basically manifestations of the data structure called circular list.

As the name itself indicates, it is a list where the nodes form a cycle. We can loosely say that the last node is connected back to the first. At any given point in time, we will have a pointer to one of the nodes in the circular list.

It is also possible that the circular list could have two pointers in each node – one points to the next and the other points to the previous – thus forming a doubly linked circular list. These types of lists are extremely common in operating systems.

The interface of a circlular linked list is same as that of a singly linked list.

It supports the following operations.

- Initialize the list
- Deinitalize the list
- Insert before/after a node
- Delete the node following the current node in the list
- Traverse
- Search for a key

The structure of the circular list is same as that of a singly linked list.
head_ : this will point to one of the nodes. We may want to give some other name for it.
The node structure is same as the one we have used in singly linked list.

```
struct node
{
        int key_;
        struct node *next_;
};
typedef struct node node_t;


struct clist
{
        node_t *head_;
};
typedef struct clist clist_t;
```

- **Initialize the list**

  The head of the circular list is grounded.
- **Deinitalize the list**

  This requires a traversal of the routine and freeing of each node. We should always delete the node which succeeds the node to which we have a pointer. This is left to the reader as an exercise to implement it.
- **Insert**

  If the list is empty, make a new node – make it point itself and set the head_ of the list to point to this node.

  If there are nodes in the list, make the next of new node take the value of next of the node to which head_ points to, change the next of head_ to point to the new node created. Effectively the node would get inserted after the node to which head points to,

```
void insert(clist_t* ptr_clist, int key)
{
        node_t *temp = create_node(key);

   if (ptr_clist->head_ == NULL)
  {
      temp->next_ = temp;
      ptr_clist->head_ = temp;
   }
   else
  {
        temp->next_ = ptr_clist->head_->next_;
        ptr_clist->head_->next_ = temp;
   }
}
```

What would happen if we change the head to point to the new node added? I leave it to you to figure out.

- **Display**

  Normally, we do not prefer do-while loop. We prefer top tested loops than bottom tested loops. But, in case of circular lists, do-while loop is preferred. We start from the head_ node, go to the next one repeatedly until the head_ is reached again.

```
void disp_list(clist_t* ptr_clist)
{
   node_t *pres = ptr_clist->head_;

   if (pres == NULL) {
      printf("List is empty\n");
      return;
   }

   node_t* temp = pres;

   printf("Circular List: ");

   do {
      printf("%d ", temp->key_);
      temp = temp->next_;
   } while (temp != pres);

   printf("\n");
}
```

Josephus Problem:

This problem has been stated in literature in number of ways. This is one of the versions.

Josephus Problem asks you to find the last remaining survivor from n people standing in a circle. Counting begins at a specific person and moves around the circle in a fixed direction. In each step, k-1 people are skipped, and the k-th person is removed. The process repeats until only one person is left.

The steps involved in solving this problem are:

1. n people are numbered from 1 to n in order and are placed in a circle
2. Starting from a designated start point, we count to kth person.
3. The kth person is eliminated
4. Counting and elimination steps are repeated until a single person is left
5. Original number of that person is the solution.

Let's take a small example:

- **n = 7** people
- **k = 3** → eliminate every 3rd person

Step	**Counting**	**Eliminated**
1	1 → 2 → **3**	**3**
2	4 → 5 → **6**	**6**
3	7 → 1 → **2**	**2**
4	4 → 5 → **7**	**7**
5	1 → 4 → **5**	**5**
6	**1** → 4 → **1**	**1**
7	4	**4**

This is an interesting exercise to program.

Applications of circular list:

- Round Robin scheduling
- Multiplayer Turn Management
- Music Playlist
- Image Slideshow
- ring-based algorithms and token passing